61CSE108: Algorithms and Data Structures

Lecture 2: Algorithm complexity analysis

Visiting lecturer: Bùi Văn Thạch

TA: Trương Quang Minh

bvthach@fit.hcmus.edu.vn,10421092@student.vgu.edu.vn

(Tentative) Schedule 2026

Date	Location	Topic
09/03	521/513 Lecture Hall	Introduction
16/03	521/513 Lecture Hall	Algorithm complexity analysis
23/03	521/513 Lecture Hall	Search
30/03	521/513 Lecture Hall	Sorting
06/04	521/513 Lecture Hall	Off because the lecturer is on business trip
13/04	521/513 Lecture Hall	Stack and Queue
20/04	521/513 Lecture Hall	Linked list
27/04	521/513 Lecture Hall	Off for Hung Kings Festival

Date	Location	Topic
04/05	521/513 Lecture Hall	Priority queue and Assignment 1
11/05	521/513 Lecture Hall	Hashing
18/05	521/513 Lecture Hall	Tree
25/05	521/513 Lecture Hall	Tree (2)
01/06	521/513 Lecture Hall	Graph
08/06	521/513 Lecture Hall	Graph (2)
15/06	521/513 Lecture Hall	Assignment 2 and Dynamic programming
22/06	521/513 Lecture Hall	Review

Course topics

1. Introduction
2. Algorithm complexity analysis
3. Search
4. Sorting
5. Stack and Queue
6. Linked list
7. Priority queue
8. Hashing
9. Tree
    1. Binary search tree (BST)
    2. AVL tree
10. Graph
    1. Graph representation
    2. Graph search
11. Algorithm designs
    1. Dynamic programming

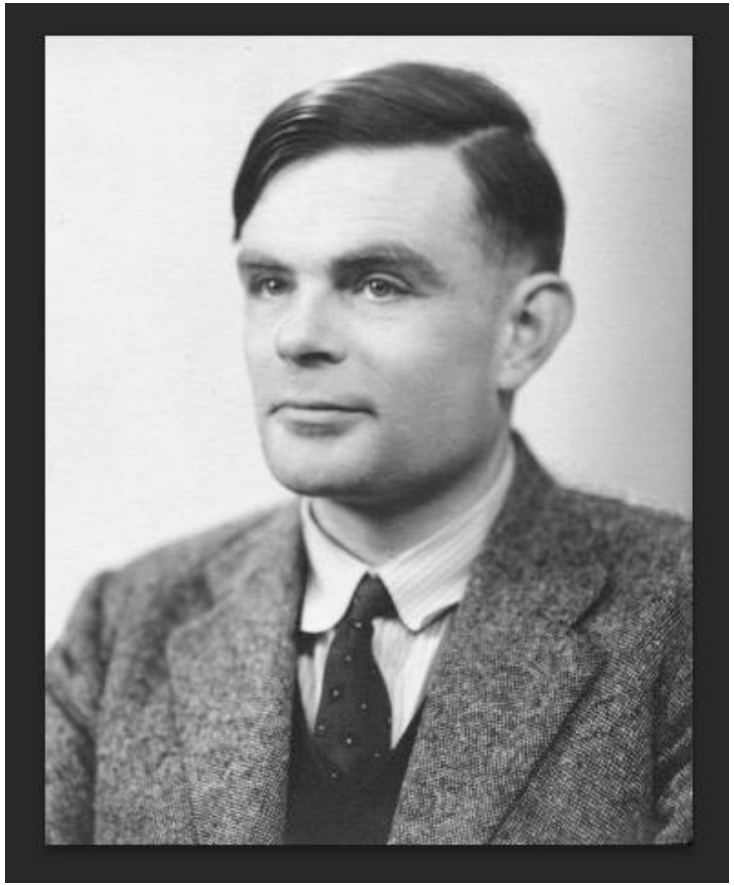
Outline

- Turing machines
- RAM model
- What is an algorithm?
- Asymptotic notations

Outline

- Turing machines
- RAM model
- What is an algorithm?
- Asymptotic notations

- The theory of computation and the practical application it made possible — the computer — was developed by an Englishman called Alan Turing.



This Photo by Unknown Author is licensed under CC BY-SA

Definition

Turing's machine — which came to be called the Turing machine — was this:

1. A tape of infinite length.
2. Finitely many squares of the tape have a single symbol from a finite language.
3. Someone (or something) that can read the squares and write in them.
4. At any time, the machine is in one of a finite number of internal states.
5. The machine has instructions that determine what it does given its internal state and the symbol it encounters on the tape. It can

– change its internal state;

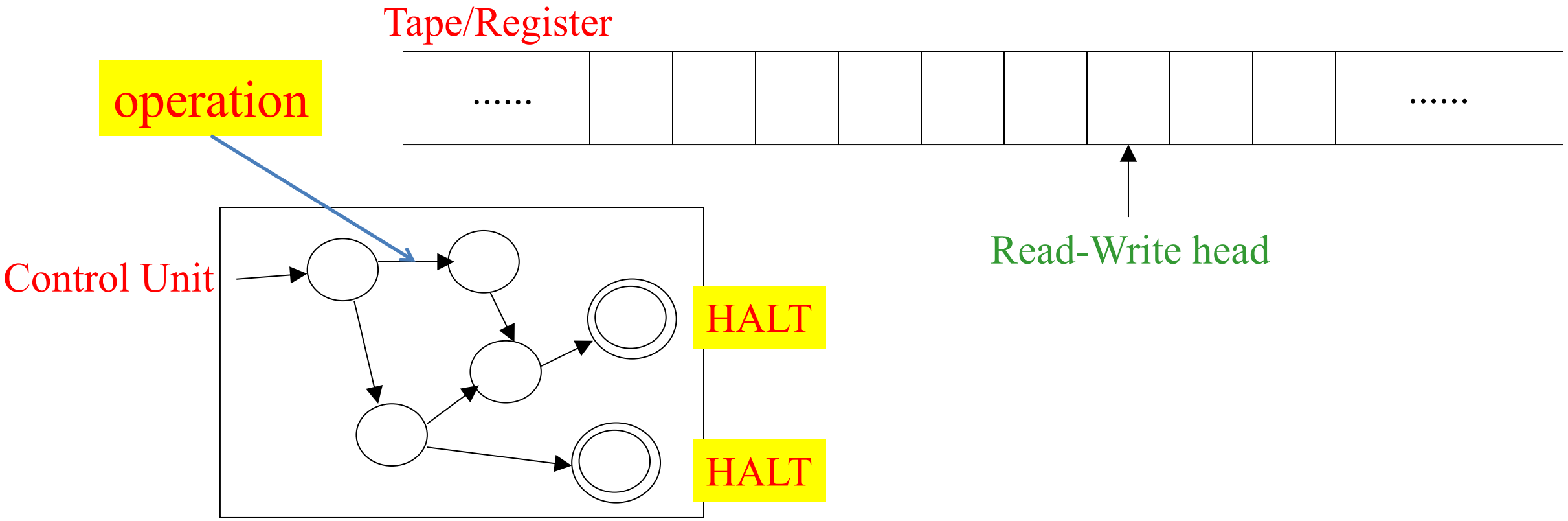
– change the symbol on the square;

– move forward;

– move backward;

– halt (i.e. stop).

- It is essential to the idea of a Turing machine that it is not a physical machine, but an abstract one — a set of procedures.



Outline

- Turing machines
- RAM model
- What is an algorithm?
- Asymptotic notations

Random-access machine (RAM) model

- Simple operations (arithmetic, comparison, conditional, etc.) each take the same, constant amount of time.

- Data stored in an infinite array of registers $(0, 1, 2, \ldots)$, each of which can hold $c \log x$ bits, where
  - x: problem size
  - c: some constant independent of x

Outline

- Turing machines
- RAM model
- What is an algorithm?
- Asymptotic notations

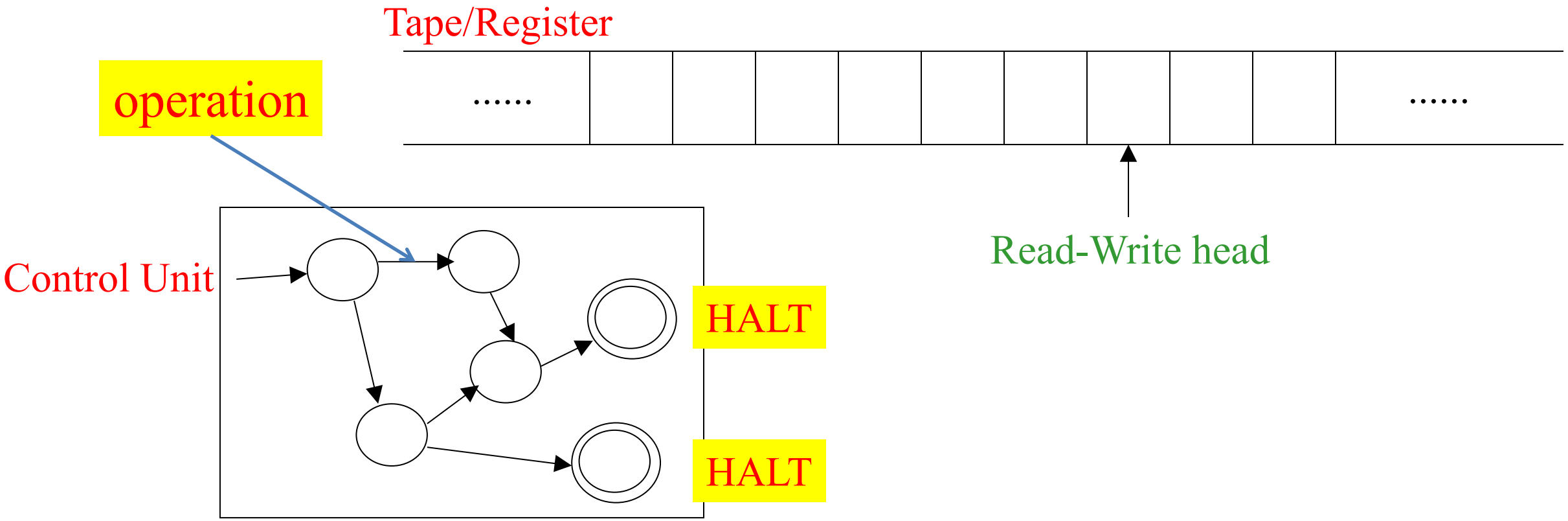
Definition

- An algorithm is a sequence of unambiguous instructions for solving a problem, that is, for obtaining a required output for any *legitimate* input in a finite amount of time.

- Analysis of algorithms is the quantitative study of the performance of algorithms, in terms of their run time, memory usage, or other properties.

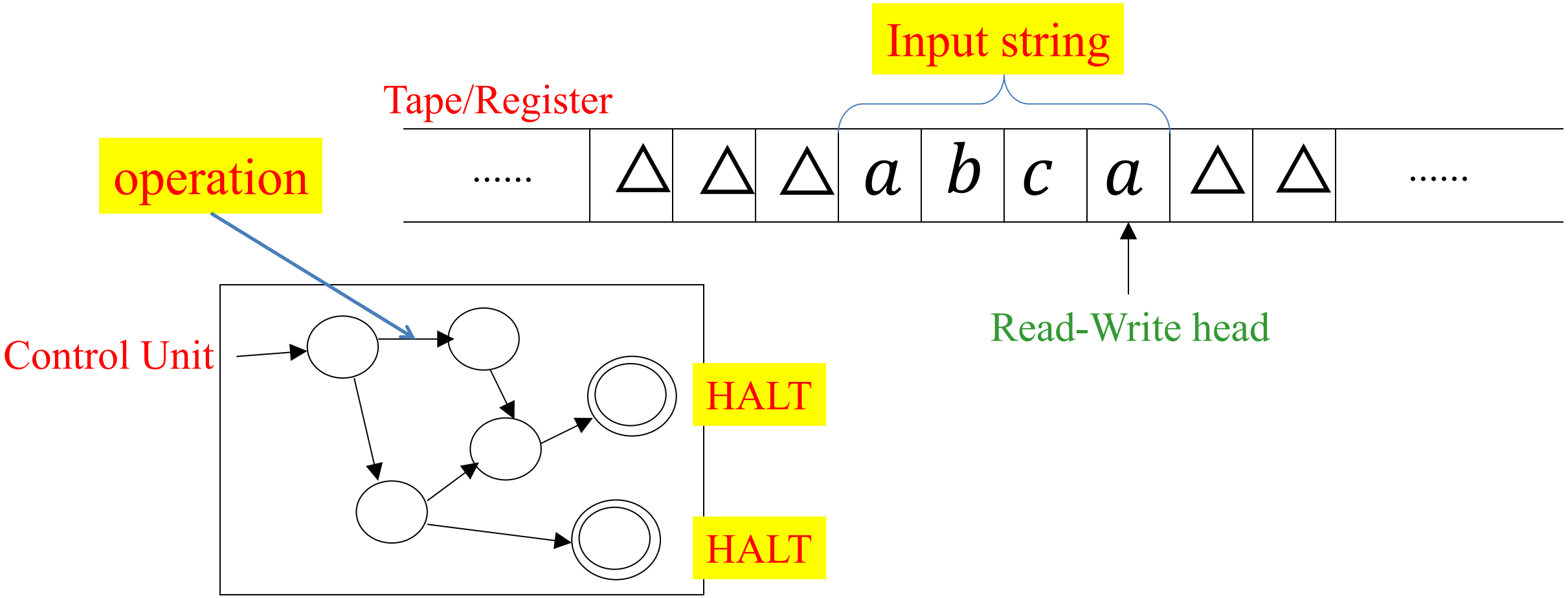
Informal descriptions

- Run time = #operations
- Memory usage = tape length



Informal descriptions

- The problem size = the length of the input string in the tape



Outline

- Turing machines
- RAM model
- What is an algorithm?
- Asymptotic notations
  - Big-Oh (O)
  - Big-Omega (Ω)
  - Big-Theta (Θ)
  - Little-Oh (o)
  - Little-Omega (ω)

- In the analysis of algorithms, we are usually interested in how the performance of our algorithm changes as the problem input size increases.

- The primary tools for measuring the growth rate of a function that describes the run time of an algorithm are the asymptotic notations.

- This provides a way of studying the algorithms themselves, independent of any specific hardware, operating system, compiler, programmer, etc.

Overview

- Let $L = \lim_{x \to \infty} \frac{f(x)}{g(x)}$

- We have:

$f < g$	$f \geq g$	$f = g$	$f \leq g$	$f > g$

	$f(x) =$	$o(g(x))$	$\Omega(g(x))$	$\Theta(g(x))$	$O(g(x))$	$\omega(g(x))$
L						
0		×			×	
$(0, \infty)$			×	×	×	
∞			×			×

Big-Oh notation (1/3): definition

- f is asymptotically bounded ABOVE by g up to constant factor c.
- Write: $f(x) = O\big(g(x)\big)$ or $f(x) \in O\big(g(x)\big)$.
- Mathematically, $\exists c > 0$ and $\exists x_0 > 0 \colon \forall x \geq x_0, |f(x)| \leq c g(x)$.
- To prove that $f(x) \in O(g(x))$, we need to provide the existence of a pair (c, x_0).

Landau, Edmund. *Handbuch der Lehre von der Verteilung der Primazahlen*. [*Handbook on the theory of the distribution of the primes*] Vol. 1. BG Teubner, 1909.

Big-Oh notation (2/3): examples

- $x^2 + 2x + 5 = O(x^2)$.

  Proof: Select $(x_0 = 1, c = 10)$ then $x^2 + 2x + 5 \leq 10x^2 \ \forall x \geq x_0$.

- $x^2 + 2x + 5 \notin O(x)$.

  Proof: Assume there exists a pair $(x_0, c > 0)$ such that $x^2 + 2x + 5 \leq cx \ \forall x \geq x_0$. Then for all $x \geq x_0$:

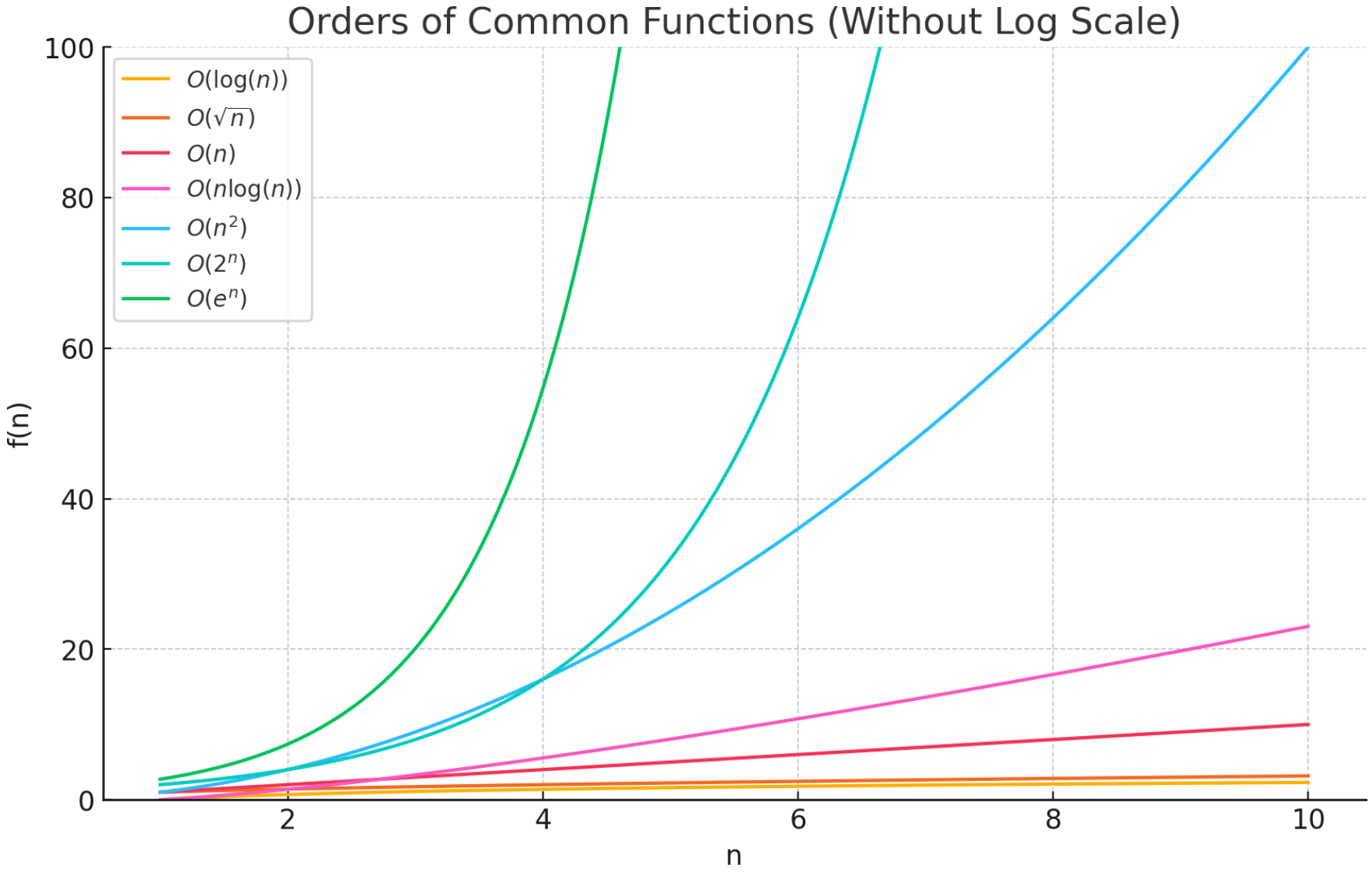
$$x^2 + (2 - c)x + 5 = \left(x - \left(1 - \frac{c}{2}\right)\right)^2 + \left(5 - \left(1 - \frac{c}{2}\right)^2\right) \leq 0$$

  This inequality does not hold when x goes to infinity.

Big-Oh notation (3/3): common functions

Notation	Name (+ function)	Example
$O(1)$	Constant	
$O(\log \log n)$	double logarithmic	
$O(\log n)$	logarithmic	
$O(\log^c n), c > 1$	polylogarithmic	
$O(n^\alpha), 0 < \alpha < 1$	fractional power	
$O(n)$	Linear	
$O(n \log n)$	Quasilinear	
$O(n^2)$	Quadratic	
$O(n^c), 1 < c$	Polynomial	
$O(c^n), c > 1$	Exponential	
$O(n!)$	factorial	

Big-Oh notation (3/3): plot of common function orders



Big-Omega notation (1/2): definition

- f is asymptotically bounded BELOW by g.
- Write: $f(x) = \Omega\big(g(x)\big)$ or $f(x) \in \Omega\big(g(x)\big)$.
- Mathematically, $\exists c > 0$ and $\exists x_0 > 0$: $\forall x \geq x_0, f(x) \geq cg(x)$.
- To prove that $f(x) \in \Omega(g(x))$, we need to provide the existence of a pair (c, x_0).
- Note that: $f(x) = \Omega\big(g(x)\big) \Leftrightarrow g(x) = O\big(f(x)\big)$.

Knuth, Donald E. "Big omicron and big omega and big theta." *ACM Sigact News* 8.2 (1976): 18-24.

Big-Omega notation (2/2): examples (1/2)

- $x^2 + 2x + 5 = \Omega(x)$.

  Proof: Select $(x_0 = 1, c = 1)$ then $x^2 + 2x + 5 \geq 1 \cdot x \ \forall x \geq x_0$.

- $x^2 + 2x + 5 = \Omega(x^2)$.

  Proof: Select $(x_0 = 1, c = 1)$ then $x^2 + 2x + 5 \geq 1 \cdot x^2 \ \forall x \geq x_0$.

Big-Omega notation (2/2): examples (2/2)

- $x^2 + 2x + 5 \notin \Omega(x^3)$.

  Proof: Assume there exists a pair $(x_0 > 0, c > 0)$ such that $x^2 + 2x + 5 \geq cx^3 \ \forall x \geq x_0$. Then for all $x \geq x_0$:

$$-\mathrm{x}^2(\mathrm{c}x - 1) + x + 5 \geq 0$$

  This inequality does not hold because when x goes to infinity, the right inequality goes to negative infinity.

Big-Theta notation (1/2): definition

- f is asymptotically bounded by g both ABOVE (with constant factor c_2) and BELOW (with constant factor c_1).
- Write: $f(x) = \Theta\big(g(x)\big)$ or $f(x) \in \Theta\big(g(x)\big)$.
- Mathematically, $\exists c_1, c_2 > 0$ and $\exists x_0 > 0: \forall x \geq x_0, c_1 g(x) \leq f(x) \leq c_2 g(x)$.
- To prove that $f(x) \in \Omega(g(x))$, we need to provide the existence of a triple (c_1, c_2, x_0).
- Note that: $f(x) = \Theta\big(g(x)\big) \Leftrightarrow g(x) = O\big(f(x)\big)$ and $f(x) = O\big(g(x)\big)$.

Knuth, Donald E. "Big omicron and big omega and big theta." *ACM Sigact News* 8.2 (1976): 18-24.

Big-Theta notation (2/2): examples (1/2)

- $x^2 + 2x + 5 = \Theta(x^2)$.

  Proof: Select $(c_1 = 1, c_2 = 10, x_0 = 1)$ then $1 \cdot x^2 \leq x^2 + 2x + 5 \leq 10x^2 \ \forall x \geq x_0$.

- $x^2 + 2x + 5 \neq \Theta(x)$.

  Proof: Assume there exists a triple $(c_1 > 0, c_2 > 0, x_0)$ such that $c_1 x \leq x^2 + 2x + 5 \leq c_2 x \ \forall x \geq x_0$. Then for all $x \geq x_0$:

  $$x^2 + (2 - c)x + 5 = \left(x - \left(1 - \frac{c_2}{2}\right)\right)^2 + \left(5 - \left(1 - \frac{c_2}{2}\right)^2\right) \leq 0$$

  This inequality does not hold when x goes to infinity.

Big-Theta notation (2/2): examples (2/2)

- $x^2 + 2x + 5 \notin \Theta(x^3)$.

  Proof: Assume there exists a pair $(x_0, c > 0)$ such that $x^2 + 2x + 5 \geq cx^3 \ \forall x \geq x_0$. Then for all $x \geq x_0$:

$$-\mathrm{x}^2(\mathrm{c}x - 1) + x + 5 \geq 0$$

  This inequality does not hold because when x goes to infinity, the right inequality goes to negative infinity.

Question (1/4)

Do these two algorithms have the same asymptotic running time?

```
int AlgA(int n) {
    int sum = 0;
    for(int i=0; i<n; i++) {
        for(int j=0; j<i; j++) {
            sum++;
        }
    }
    return sum;
}
```

```
int AlgB(int n) {
    int sum = 0;
    for(int i=0; i<n; i++) {
        for(int j=0; j<n; j++) {
            sum++;
        }
    }
    return sum;
}
```

Question (2/4)

- In an iterative algorithm, let a_i be the number of operations, e.g, comparisons and assignments, at iteration i.
- A common tool for analyzing the iterative algorithms is the summation:

$$\sum_{i=\ell}^{n} a_i = a_\ell + a_{\ell+1} + \cdots + a_{n-1} + a_n$$

- If the upper limit is infinite, we interpret this as an implicit limit:

$$\sum_{i=\ell}^{\infty} a_i = \lim_{n\to\infty} \sum_{i=\ell}^{n} a_i$$

Question (3/4)

$$A(n) = \sum_{i=0}^{n-1} \sum_{j=0}^{i-1} 1 = \sum_{i=0}^{n-1} i = \frac{n(n-1)}{2}$$
$$= \Theta(n^2)$$

$$B(n) = \sum_{i=0}^{n-1} \sum_{j=0}^{n-1} 1 = \sum_{i=0}^{n-1} n = n^2$$
$$= \Theta(n^2)$$

```
int AlgA(int n) {
    int sum = 0;
    for(int i=0; i<n; i++) {
        for(int j=0; j<i; j++) {
            sum++;
        }
    }
    return sum;
}
```

```
int AlgB(int n) {
    int sum = 0;
    for(int i=0; i<n; i++) {
        for(int j=0; j<n; j++) {
            sum++;
        }
    }
    return sum;
}
```

Question (4/4)

Do these two algorithms have the same asymptotic running time?

➔Yes, the run times are both $\Theta(n^2)$.

```
int AlgA(int n) {
    int sum = 0;
    for(int i=0; i<n; i++) {
        for(int j=0; j<i; j++) {
            sum++;
        }
    }
    return sum;
}
```

```
int AlgB(int n) {
    int sum = 0;
    for(int i=0; i<n; i++) {
        for(int j=0; j<n; j++) {
            sum++;
        }
    }
    return sum;
}
```

Exercise 1

Do these two algorithms have the same asymptotic running time?

```
int AlgC(int n) {
    int sum = 0;
    for(int i=1; i<=n; i*=2) {
        for(int j=0; j<i; j++) {
            sum++;
        }
    }
    return sum;
}
```

```
int AlgD(int n) {
    int sum = 0;
    for(int i=1; i<=n; i*=2) {
        for(int j=0; j<n; j++) {
            sum++;
        }
    }
    return sum;
}
```

Solution

- NO, the asymptotic run times are different. Observe that i is always a power of 2. Let $i = 2k$, so that $k = \log_2 n$.
- $C(n) = \sum_{k=0}^{\lceil \log_2 n \rceil - 1} \sum_{j=0}^{2^k - 1} 1 = \sum_{k=0}^{\lceil \log_2 n \rceil - 1} 2^k = 2^{\lceil \log_2 n \rceil} - 1 = \Theta(n)$
- $D(n) = \sum_{k=0}^{\lceil \log_2 n \rceil - 1} \sum_{j=0}^{n-1} 1 = \sum_{k=0}^{\lceil \log_2 n \rceil - 1} n = n\lceil \log_2 n \rceil = \Theta(n \ln n)$

Exercise 2

What is the running time of power?

```
double power(double a, long n)
{
    //@requests n>=0
    //@assures result=a^n
    double result=1;
    double x = a;
    long y = n;

    while (y) {
        if (y%2 == 1)
            result *=x;
        x *=x;
        y /=2;
    }
    return result;
}
```

Little-Oh notation (1/2): definition

- f is asymptotically dominated by g (for ANY constant factor c).
- Write: $f(x) = o\big(g(x)\big)$ or $f(x) \in o\big(g(x)\big)$.
- Mathematically, $\forall c > 0$ and $\exists x_0 > 0 : \forall x \geq x_0, f(x) \leq c g(x)$.
- To prove that $f(x) \in o(g(x))$, we need to provide the existence of x_0 for every $c > 0$.
- Note that: $f(x) = o\big(g(x)\big) \Leftrightarrow \lim_{x \to \infty} \frac{f(x)}{g(x)} = 0$.

Landau, Edmund. *Handbuch der Lehre von der Verteilung der Primazahlen*. [*Handbook on the theory of the distribution of the primes*] Vol. 1. BG Teubner, 1909.

Little-Oh notation (2/2): example

- $x^2 + 2x + 5 = o(x^3)$.

  Proof: $\lim_{x\to\infty} \frac{f(x)}{g(x)} = \lim_{x\to\infty} \frac{x^2+2x+5}{x^3} = \lim_{x\to\infty} \left(\frac{1}{x} + \frac{2}{x^2} + \frac{5}{x^3}\right) = 0.$

- $x^2 + 2x + 5 \notin o(x)$.

  Proof: $\lim_{x\to\infty} \frac{f(x)}{g(x)} = \lim_{x\to\infty} \frac{x^2+2x+5}{x} = \lim_{x\to\infty} \left(x + 2 + \frac{5}{x}\right) = \infty.$

Little-Omega notation (1/2): definition

- f asymptotically dominates g.
- Write: $f(x) = \omega\big(g(x)\big)$ or $f(x) \in \omega\big(g(x)\big)$.
- Mathematically, $\forall c > 0$ and $\exists x_0 > 0 : \forall x \geq x_0, f(x) \geq cg(x)$.
- To prove that $f(x) \in o(g(x))$, we need to provide the existence of x_0 for every $c > 0$.
- Note that: $f(x) = \omega\big(g(x)\big) \Leftrightarrow \lim_{x \to \infty} \frac{f(x)}{g(x)} = \infty$.

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to algorithms*. MIT press.

Little-Oh notation (2/2): example

- $x^2 + 2x + 5 = \omega(x)$.

Proof: $\lim_{x\to\infty} \frac{f(x)}{g(x)} = \lim_{x\to\infty} \frac{x^2+2x+5}{x} = \lim_{x\to\infty} \left(x + 2 + \frac{5}{x}\right) = \infty$.

- $x^2 + 2x + 5 \notin o(x^3)$.

Proof: $\lim_{x\to\infty} \frac{f(x)}{g(x)} = \lim_{x\to\infty} \frac{x^2+2x+5}{x^3} = \lim_{x\to\infty} \left(\frac{1}{x} + \frac{2}{x^2} + \frac{5}{x^3}\right) = 0$.

Exercises (1/5)

Prove the following statements:

1. $n^3 + 1000n^2 = O(n^4)$.
2. $\log n = O(n)$.
3. $\log n = O(\sqrt{n})$.
4. $n! \notin O(n^c)$ for any positive constant c.
5. $n^a = O(b^n)$ for any positive constants a and $b > 1$.
6. $\log n! = O(n \log n)$ and $\log n! \geq \frac{n}{2} \log \frac{n}{2}$ to get $\log n! = \Theta(n \log n)$.
7. $1000x^3 - x^2 + 79 = \Theta(x^3)$.

Exercises (2/5)

8. Let $f(x) = \mathrm{O}\big(h(x)\big)$ and $g(x) = \mathrm{O}\big(h(x)\big)$. Let $a, b > 0$. Prove that $af(x) + bg(x) = \mathrm{O}\big(h(x)\big)$.
9. Let $f_1(x) = \mathrm{O}\big(g_1(x)\big)$ and $f_2(x) = \mathrm{O}\big(g_2(x)\big)$. Prove that $f_1(x)f_2(x) = \mathrm{O}\big(g_1(x)g_2(x)\big)$.
10. Prove that $x^n + a_{n-1}x^{n-1} + \cdots + a_0 = \mathrm{O}(x^n)$.
11. Prove that $x^n + a_{n-1}x^{n-1} + \cdots + a_0 = \Theta(x^n)$.
12. $\log n = O(n^c)$ where $0 < c < 1$.

Exercises (3/5)

- How many comparisons, and assignments are there in the following code fragment with the size n?

```
sum = 0;
for (i = 0; i < n; i++)
{
    cin >> x;
    sum = sum + x;
}
```

Exercises (4/5)

How many assignments are there in the following code fragment with the size *n*?

```
for (i = 0; i < n ; i++)
      for (j = 0; j < n; j++)
      {
            C[i][j] = 0;
            for (k = 0; k < n; k++)
                  C[i][j] = C[i][j] + A[i][k]*B[k][j];
      }
```

Exercises (5/5)

- Give the order of growth (as a function of N) of the running time of the following code fragment:

```
int sum = 0;
for (int n = N; n > 0; n /= 2)
   for (int i = 0; i < n; i++)
      sum++;
```

Q & A